Project Title

Symptom-to-Diagnosis Matcher

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
P. Nirvigna (2520090144)
Rohita (2520090154)

Team Number: 29

Section: 12

Under the guidance of

Ms.K.Chandusha

Asst. Prof., CS&IT

(DEEMED TO BE UNIVERSITY)
BACHUPALLY
Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING /

COMPUTER SCIENCE AND INFORMATION

TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE

DECLARATION

The Mini Project Report entitled **"Symptom-to-Diagnosis Matcher"** is a record of Bonafide work of **P.Nirvighna sai - 2520090144, Rohita - 2520090154,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

P.Nirvighna sai – 2520090144

Rohita - 2520090154

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **"Symptom-to-Diagnosis Matcher"** is a bonafide work done and submitted by **P.Nirvighna sai - 2520090144, Rohita – 2520090154** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, <<Guide Name>>, <<Designation>>, <Department>> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

P.Nirvighna sai – 2520090144

Rohita - 2520090154

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	11
7.	Results & Discussion	12
8.	Conclusion & Future Scope	13
9.	References	14

1.ABSTRACT

The Symptom-to-Diagnosis Matcher is a clinical decision-support system designed to convert free-text patient symptoms into ranked lists of probable medical conditions. The proposed engine uses natural language processing (NLP) to normalize symptom descriptions by handling spelling errors, abbreviations, synonyms, and variations in expression. Extracted symptoms are matched against a structured medical knowledge base containing diseases, associated symptoms, and diagnostic test information. A transparent weighted scoring mechanism evaluates symptom relevance, frequency, and importance to generate confidence scores for each candidate diagnosis. The system also recommends a minimal set of discriminating diagnostic tests that can effectively distinguish among the top-ranked conditions, reducing unnecessary investigations and improving efficiency. By combining NLP, knowledge-based reasoning, interpretable scoring, and intelligent test recommendation, the proposed approach supports accurate preliminary diagnosis, assists healthcare professionals in clinical triage, and enhances decision making in telemedicine, primary healthcare screening, and medical education environments.

Team Name:
Symptom-to-Diagnosis Matcher

Team Logo (if any):

Team Members:

P.Nirvighna sai – 2520090144

Rohita - 2520090154

2. INTRODUCTION

People often describe their symptoms in everyday language rather than using medical terms. Different people may use different words, abbreviations, or spellings to describe the same symptom. Matching these descriptions with possible medical conditions requires text normalization, a structured knowledge base, and a clear ranking method.

MediMatch (Symptom-to-Diagnosis Matcher) is a Java and Data Structures and Algorithms-based project that accepts free-text symptoms, processes them, and compares them with a sample medical knowledge base. It uses symptom normalization, duplicate removal, weighted scoring, and sorting to generate ranked educational matches. The system also suggests example follow-up assessments related to the entered symptoms.

The main purpose of MediMatch is to demonstrate how DSA concepts can be applied to healthcare-related information processing. It can serve as an educational tool for understanding preliminary symptom matching and clinical decision-support workflows. The system is not a substitute for professional medical advice or diagnosis.

3. LITERATURE SURVEY

WebMD Symptom Checker

WebMD helps users explore possible health conditions based on their symptoms. It provides health information in an accessible format. MediMatch follows a similar concept by matching user-entered symptoms with possible conditions.

Mayo Clinic

Mayo Clinic provides information about diseases, symptoms, causes, and treatments. It demonstrates the importance of connecting symptoms with structured medical information. MediMatch uses a sample knowledge base to match symptoms with possible conditions.

NHS Health A to Z

NHS Health A to Z organizes information about different health conditions and their symptoms. It helps users find relevant health information easily. MediMatch applies a similar concept by organizing symptoms and conditions for efficient matching.

MedlinePlus

MedlinePlus provides medical information about diseases, symptoms, and diagnostic tests. Its structured health resources are useful for understanding relationships between symptoms and medical conditions. MediMatch uses a simplified knowledge base to generate educational matches.

Isabel Healthcare

Isabel Healthcare provides clinical decision-support tools that help healthcare professionals consider possible diagnoses. It demonstrates the use of computer-based support for evaluating medical information. MediMatch implements a simplified approach using symptom matching, weighted scoring, and ranking.

PubMed

PubMed is a biomedical research database containing scientific articles related to medicine and healthcare. It helps researchers explore existing approaches to medical information processing and clinical decision support. It can serve as a reference for understanding the research background of MediMatch.

4. Problem Statement and Objectives

Problem Statement: To develop a Java-based symptom-matching system that converts free-text symptoms into ranked, explainable matches with possible medical conditions using Data Structures and Algorithms.

Problem Description: Patients describe symptoms in everyday language, while medical information is stored in structured formats. MediMatch bridges this gap by normalizing symptoms, matching them with a sample knowledge base, ranking possible conditions, and suggesting follow-up assessments.

Objectives:

1. Accept symptom descriptions in everyday language.
2. Normalize common phrases, synonyms, abbreviations, and variations.
3. Remove duplicate symptoms using suitable data structures.
4. Match recognized symptoms with a structured knowledge base.
5. Calculate weighted scores and rank matching conditions.
6. Suggest a small set of relevant follow-up assessments.
7. Apply HashMap, HashSet, sorting, and greedy selection.
8. Display understandable educational results through a user-friendly website

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 or equivalent.
- RAM: Minimum 4 GB.
- Storage: Minimum 1 GB of free space.
- Device: Laptop or desktop computer.

Software Requirements:

- **Operating System:** Windows 10/11 or Linux.
- **Programming Language:** Java.
- **Web Technologies:** HTML, CSS, and JavaScript.
- **Algorithm Tools:** Implementations of string algorithms, dynamic programming, network flow, approximation, randomized, and parallel algorithms.

6. IMPLEMENTATION

```
    for (int i = 0; i <= n - m; i++) {
        int j = 0;

        while (j < m &&
               text.charAt(i + j) == pattern.charAt(j)) {
            j++;
        }

        if (j == m)
            return i;
    }
    return -1;
}

public static void main(String[] args) {
    String symptoms = "fever cough headache";
    String keyword = "cough";

    System.out.println(
        "Position: " + search(symptoms, keyword)
    );
}

    for (int i = 0; i <= n; i++)
        dp[i][0] = i;

    for (int j = 0; j <= m; j++)
        dp[0][j] = j;

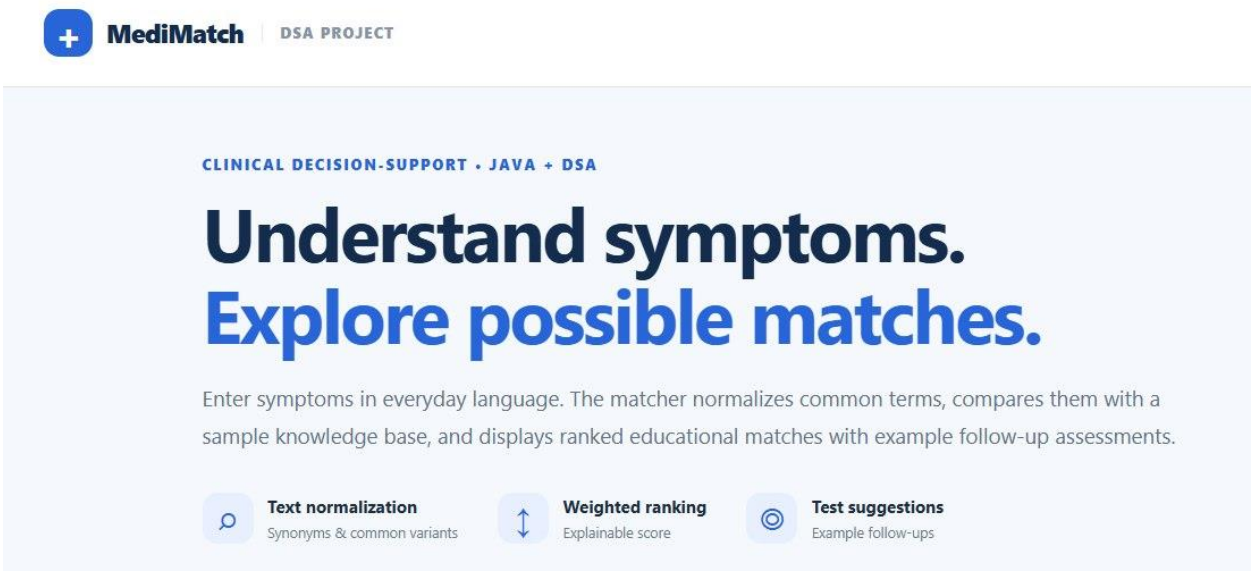
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int cost = a.charAt(i - 1)
                    == b.charAt(j - 1) ? 0 : 1;

            int delete = dp[i - 1][j] + 1;
            int insert = dp[i][j - 1] + 1;
            int replace = dp[i - 1][j - 1] + cost;

            dp[i][j] = Math.min(
                delete, Math.min(insert, replace)
            );
        }
    }
    return dp[n][m];
}

public static void main(String[] args) {
    System.out.println(
        "Edit distance: " +
        distance("headake", "headache")
    );
}
```

7. RESULTS & DISCUSSION



MediMatch demonstrates how DSA concepts can organize symptom information and generate ranked matches. The results depend on the available sample knowledge base and implemented scoring rules. The system is an educational prototype and does not provide clinically validated diagnoses.

8. CONCLUSION & FUTURE SCOPE

MediMatch demonstrates the application of Data Structures and Algorithms in a healthcare-related web application. It processes free-text symptoms through normalization, duplicate removal, condition matching, weighted scoring, and sorting. It also demonstrates how relevant follow-up assessments can be suggested using a greedy selection approach.

The project combines a simple user interface with explainable algorithmic processing. It demonstrates the practical use of HashMap, HashSet, sorting, and greedy selection in a symptom-matching application. The current prototype uses a sample knowledge base and is intended for educational purposes rather than confirmed medical diagnosis.

FUTURE SCOPE

Expand the knowledge base with medically reviewed symptom and condition information.

Improve symptom matching using advanced NLP and spelling correction.

Enhance condition ranking by considering symptom severity, duration, and importance.

Implement a database, multilingual input, and a mobile-friendly interface for better accessibility.

9. REFERENCES

1. **WebMD, Symptom Checker — https://www.webmd.com/symptoms**
2. **Mayo Clinic, Diseases and Conditions — https://www.mayoclinic.org/diseases-conditions**
3. **NHS, Health A to Z — https://www.nhs.uk/health-a-to-z/**
4. **MedlinePlus — https://medlineplus.gov/**
5. **Isabel Healthcare — https://www.isabelhealthcare.com/**
6. **PubMed, U.S. National Library of Medicine — https://pubmed.ncbi.nlm.nih.gov/**
7. **Oracle Java Documentation — https://docs.oracle.com/en/java/**
8. **Java Collections Framework — https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html**
9. **MDN Web Docs, HTML — https://developer.mozilla.org/en-US/docs/Web/HTML**
10. **MDN Web Docs, CSS — https://developer.mozilla.org/en-US/docs/Web/CSS**